Scientific Calculator using DevOps Pipeline

Nitheesh Vemula
MT2025079

GitHub Repository:
https://github.com/VemulaNitheesh/Scientific-calculator-devOps

Docker Hub Repository:
https://hub.docker.com/r/nitheesh05/calculator

Introduction

DevOps is a combination of Development and Operations which aims to automate the software development lifecycle. It helps teams to build, test and deploy applications faster and more reliably.

In this project a Scientific Calculator application developed in Java is deployed using a DevOps pipeline consisting of GitHub, Jenkins, Docker and Ansible.

What and Why of DevOps

DevOps integrates development and operations teams to improve collaboration and automate the software delivery process.

Benefits of DevOps include:

- Faster deployment
- Continuous integration and delivery
- Reduced manual errors
- Better monitoring

Tools Used

- GitHub – Source code management
- Jenkins – Continuous Integration and Continuous Deployment
- Maven – Build tool
- Docker – Containerization
- Docker Hub – Image repository
- Ansible – Automated deployment

DevOps Workflow

The workflow of the project is as follows:

Developer writes code → Pushes code to GitHub → Jenkins builds the project → Docker containerizes the application → Docker Hub stores the image → Ansible deploys the application → Monitoring through logs.

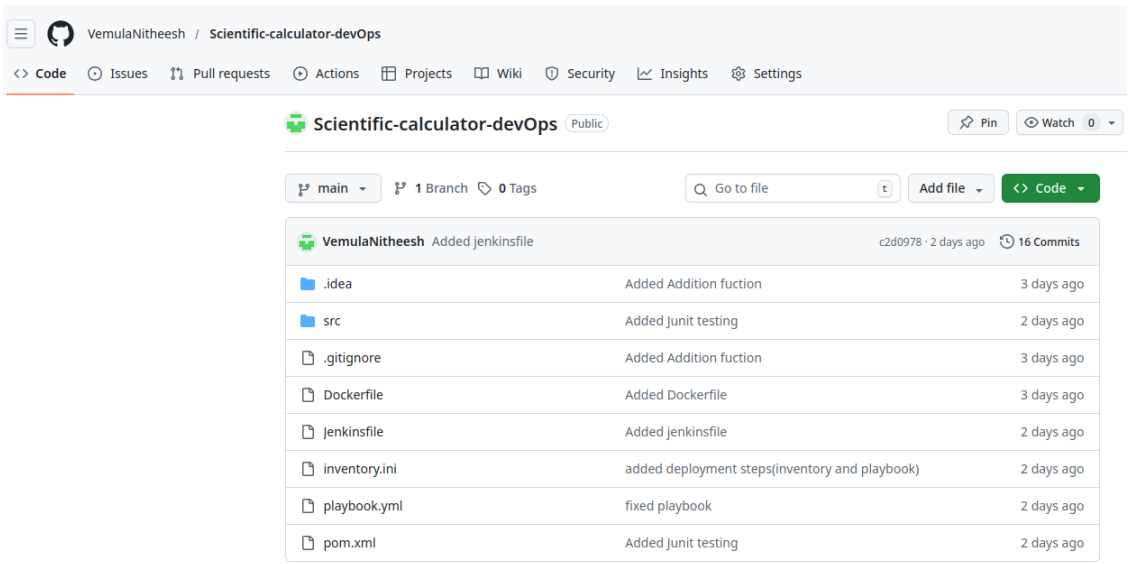
Step 1: Source Control Management using GitHub

GitHub is used to store the source code and manage versions of the application. Developers push code to the repository and Jenkins automatically fetches the code from GitHub.

Repository link:

`https://github.com/VemulaNitheesh/Scientific-calculator-devOps`

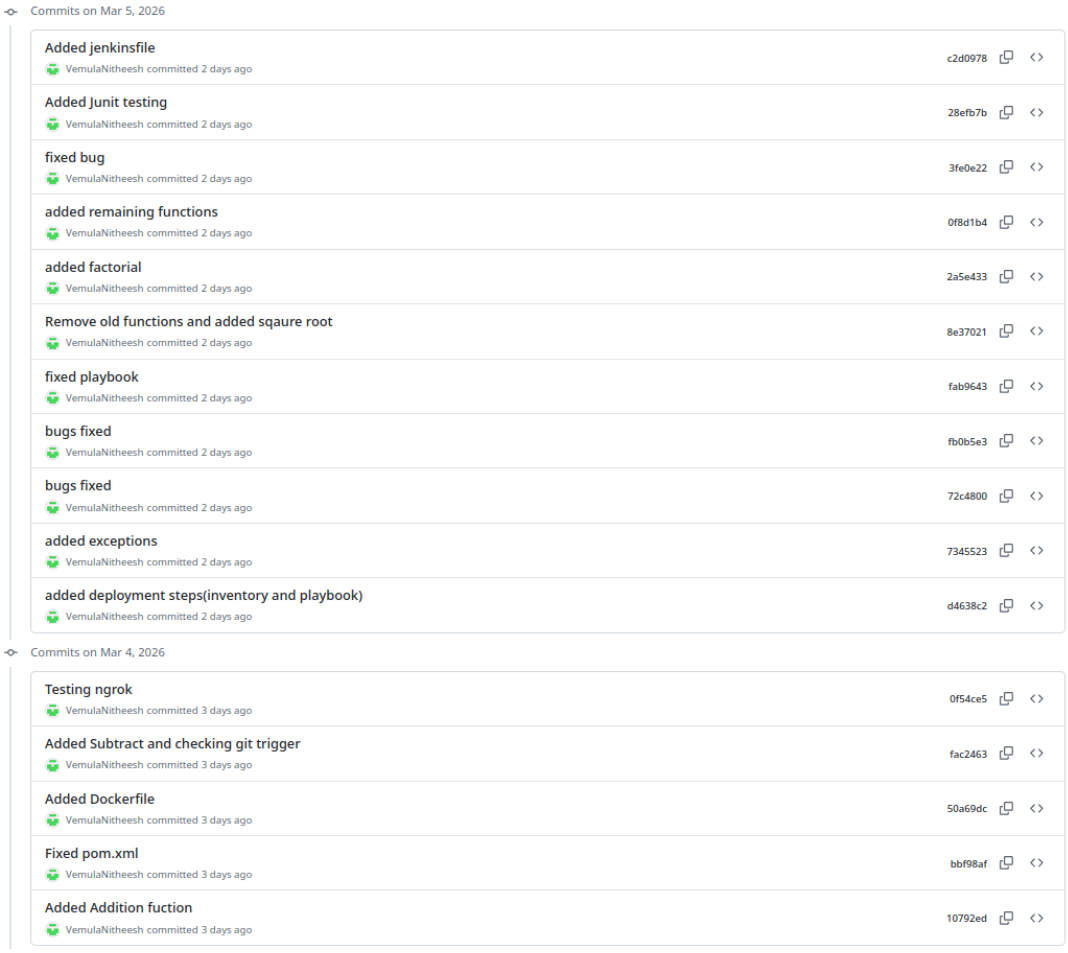
The repository was created in GitHub and the project files were uploaded.



After creating the repository, the following Git commands were used to push the code.

```
git init
git add .
git commit -m "Initial commit"
git remote add origin https://github.com/VemulaNitheesh/Scientific-calculator
git push origin main
```

The commit history of the project can be seen in the GitHub interface.



Step 2: Build Automation using Maven

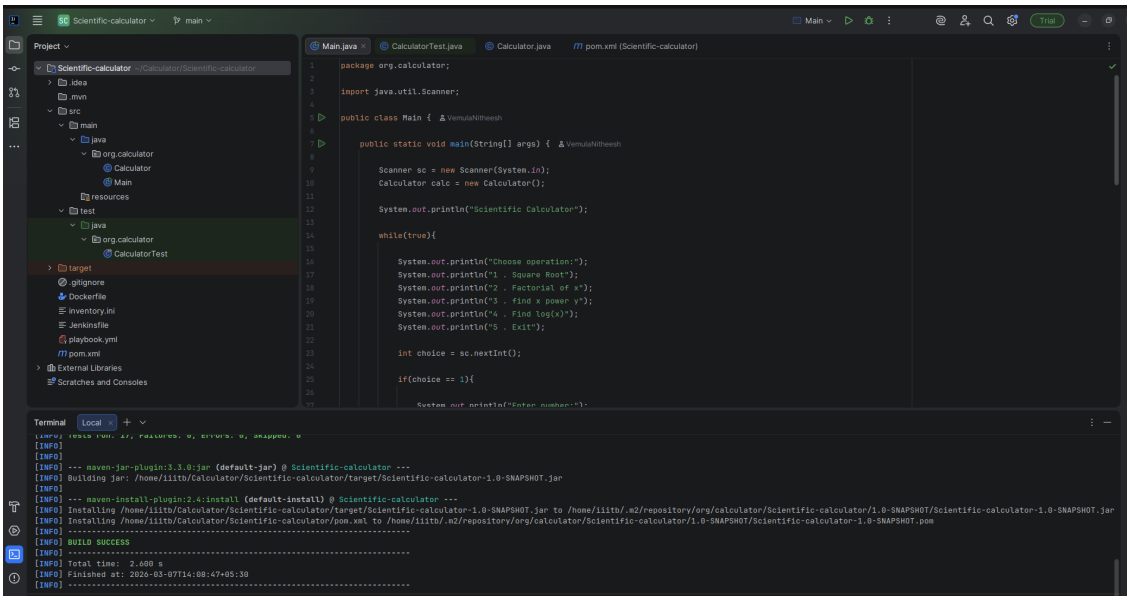
Maven is used as the build automation tool for compiling the Java application and packaging it into a JAR file.

The project contains a **pom.xml** file which defines dependencies and build configurations.

The project was built using the following command.

```
mvn clean package or mvn clean install
```

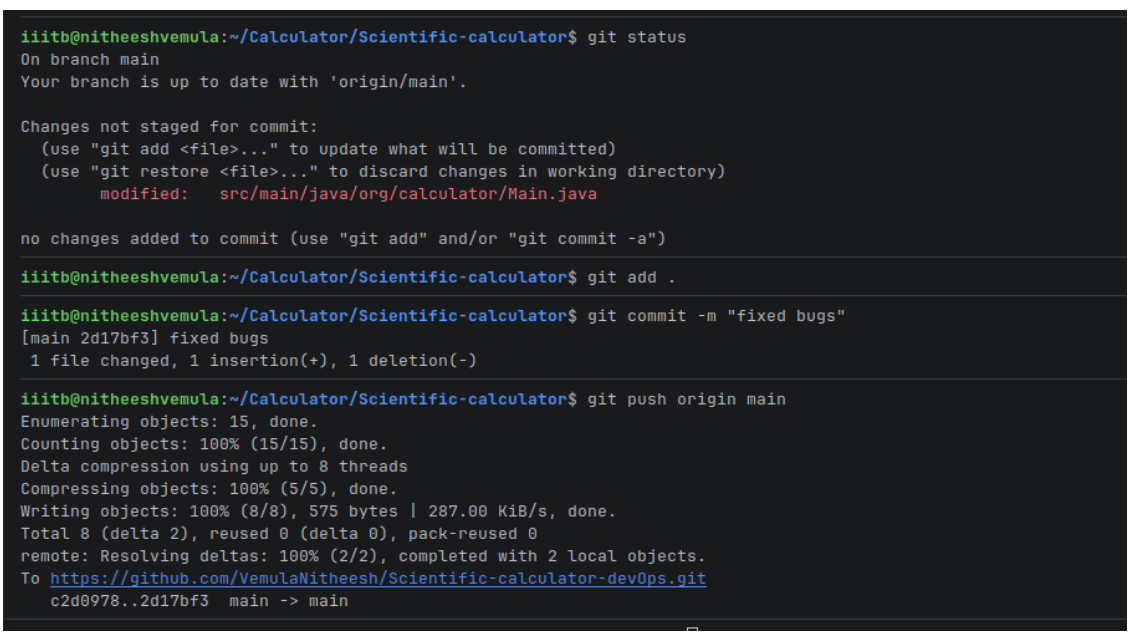
This command compiles the source code and generates a JAR file in the target directory.



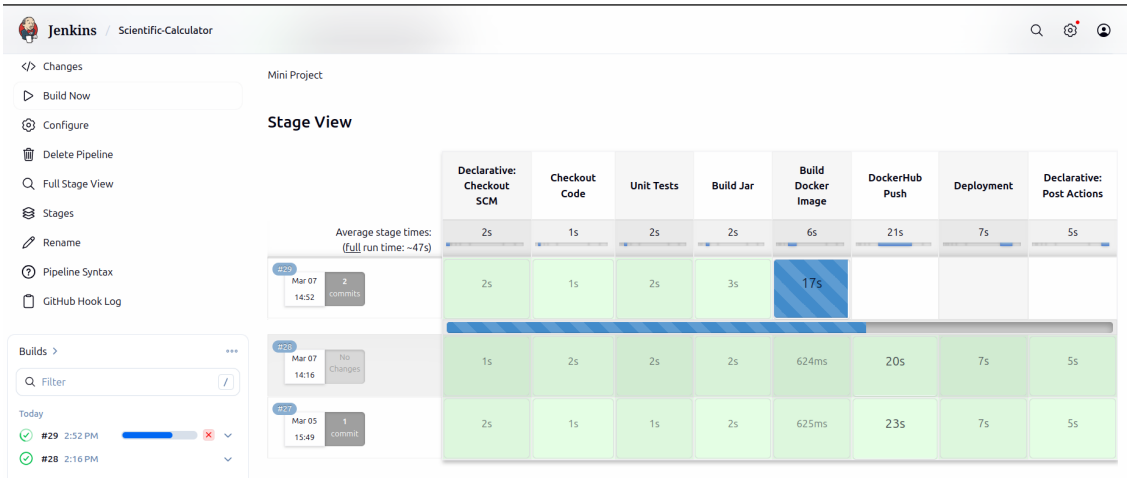
Continuous Integration using Jenkins

Jenkins is used to automate the build and deployment process. Whenever new code is pushed to GitHub, Jenkins automatically triggers the pipeline.

The Jenkins pipeline is connected to the GitHub repository using GitHub webhooks so that every push triggers a new build.



This is the Jenkins stage view showing all pipeline stages executed during the build process.(Triggered by GitHub push)



A Jenkins pipeline was created to perform the following steps:

- Checkout SCM from GitHub
- Fetch latest source code
- Perform unit testing
- Build the JAR file using Maven
- Build Docker image
- Push image to Docker Hub
- Deploy the application
- Send email notification after build completion

After the pipeline execution, Jenkins sends an email notification indicating whether the build was successful or failed.

Step 4: Containerization using Docker

Docker is used to package the application and its dependencies into a container.

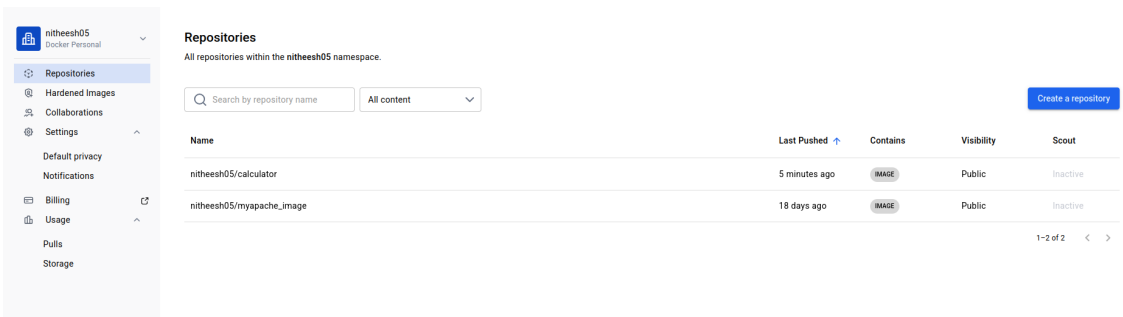
Docker images built by different builds:

```
iiitb@nitheeshvemula:~$ docker images
REPOSITORY                  TAG       IMAGE ID       CREATED         SIZE
nitheesh05/calculator       latest    0a7134ccc6cc   2 minutes ago   448MB
nitheesh05/calculator       <none>    93ceaa1f0379   46 hours ago    448MB
nitheesh05/calculator       <none>    49731b41beb7   47 hours ago    448MB
nitheesh05/calculator       <none>    d0b0d53ff374   47 hours ago    448MB
nitheesh05/calculator       <none>    0988ab5f13bb   47 hours ago    448MB
```

Docker running container:

```
iiitb@nitheeshvemula: ~
iiitb@nitheeshvemula:~$ docker ps
CONTAINER ID   IMAGE                          COMMAND                  CREATED          STATUS          PORTS     NAMES
01bc22a6a833   nitheesh05/calculator:latest   "/__cacert_entrypoin…"   11 seconds ago   Up 10 seconds             calculator-container
iiitb@nitheeshvemula:~$
```

After building the image, it was pushed to Docker Hub.



Step 5: Deployment using Ansible

Ansible is used for automating the deployment of the application to remote servers.

An inventory file was created to define the target servers.

The Ansible playbook used for deployment.

The deployment was executed using the following command.

```
ansible-playbook playbook.yml -i inventory.ini
```

```
iiitb@nitheeshvemula:~$ docker exec -it calculator-container bash
root@01bc22a6a833:/app# ls
calculator.jar
root@01bc22a6a833:/app#
root@01bc22a6a833:/app#
```

Scientific Calculator Application

The deployed application performs various mathematical operations. Operations performed by the calculator are shown below.

```
root@01bc22a6a833:/app# java -jar calculator.jar
Scientific Calculator
Choose operation:
1 . Square Root
2 . Factorial of x
3 . find x power y
4 . Find log(x)
5 . Exit
1
Enter number:
10
Result = 3.1622776601683795
Choose operation:
1 . Square Root
2 . Factorial of x
3 . find x power y
4 . Find log(x)
5 . Exit
2
Enter x:
5
Factorial is 120
Choose operation:
1 . Square Root
2 . Factorial of x
3 . find x power y
4 . Find log(x)
5 . Exit
3
Enter base:
3
Enter exponent:
4
Result = 81.0
Choose operation:
1 . Square Root
2 . Factorial of x
3 . find x power y
4 . Find log(x)
5 . Exit
4
Enter number :
9
Result :2.1972245773362196
Choose operation:
1 . Square Root
2 . Factorial of x
3 . find x power y
4 . Find log(x)
5 . Exit
5
Exiting calculator...
root@01bc22a6a833:/app# exit
```

Conclusion

The project demonstrates the implementation of a DevOps pipeline for automating the development and deployment of a scientific calculator application. By integrating GitHub, Jenkins, Docker and Ansible, the entire lifecycle from code commit to deployment is automated, making the system more efficient and reliable.